

Circuit-SAT and Polynomial Reductions

Question–Answer Notes for Exercises 34.3-1 to 34.3-8

Background

Main Tools

These exercises use three core ideas:

- Boolean circuits are satisfiable if some input assignment makes the output 1.
- Polynomial-time reductions compose.
- Completeness means every language in the class reduces to the target language.

1 Exercise 34.3-1

Question 34.3-1

Verify that the circuit in Figure 34.8(b) is unsatisfiable.

Answer

The circuit in Figure 34.8(b) corresponds to the Boolean expression

$$((x_1 \vee x_2) \wedge \neg(\neg x_3)) \wedge (\neg(\neg x_3) \vee (x_1 \wedge \neg x_3 \wedge x_2)) \wedge (x_1 \wedge \neg x_3 \wedge x_2).$$

Canceling the double negations gives

$$((x_1 \vee x_2) \wedge x_3) \wedge (x_3 \vee (x_1 \wedge \neg x_3 \wedge x_2)) \wedge (x_1 \wedge \neg x_3 \wedge x_2).$$

Why It Is Unsatisfiable

The first conjunct

$$(x_1 \vee x_2) \wedge x_3$$

can be true only if

$$x_3 = 1.$$

The last conjunct

$$x_1 \wedge \neg x_3 \wedge x_2$$

can be true only if

$$x_3 = 0.$$

The whole circuit output is an AND of these conjuncts, so both would have to be true at the same time. That would require both $x_3 = 1$ and $x_3 = 0$, which is impossible. Therefore the circuit is unsatisfiable.

2 Exercise 34.3-2

Question 34.3-2

Show that the polynomial-time reduction relation is transitive. That is, prove that if

$$L_1 \leq_p L_2 \quad \text{and} \quad L_2 \leq_p L_3,$$

then

$$L_1 \leq_p L_3.$$

Answer

Compose the two reduction functions.

Proof

Since $L_1 \leq_p L_2$, there is a polynomial-time computable function f_1 such that

$$x \in L_1 \iff f_1(x) \in L_2.$$

Since $L_2 \leq_p L_3$, there is a polynomial-time computable function f_2 such that

$$z \in L_2 \iff f_2(z) \in L_3.$$

Define

$$g(x) = f_2(f_1(x)).$$

Then g is polynomial-time computable because polynomial-time functions compose. Also,

$$x \in L_1 \iff f_1(x) \in L_2 \iff f_2(f_1(x)) \in L_3 \iff g(x) \in L_3.$$

Therefore $L_1 \leq_p L_3$.

3 Exercise 34.3-3

Question 34.3-3

Prove that

$$L \leq_p \bar{L} \iff \bar{L} \leq_p L.$$

Answer

The same reduction function works in both directions.

Proof

Suppose $L \leq_p \bar{L}$ by a polynomial-time computable function f . Then for every x ,

$$x \in L \iff f(x) \in \bar{L}.$$

Taking complements of both sides gives

$$x \notin L \iff f(x) \notin \bar{L}.$$

Equivalently,

$$x \in \bar{L} \iff f(x) \in L.$$

So the same function f proves

$$\bar{L} \leq_p L.$$

The reverse direction is identical. Therefore

$$L \leq_p \bar{L} \iff \bar{L} \leq_p L.$$

4 Exercise 34.3-4

Question 34.3-4

Show that we could have used a satisfying assignment as a certificate in an alternative proof of Lemma 34.5. Which certificate makes for an easier proof?

Answer

Yes. Instead of using values for all wires, the certificate can be just an assignment to the input variables of the circuit.

Alternative Verifier

Given a circuit C and an assignment to its input variables:

1. evaluate the gates of C in topological order, from inputs toward the output;
2. compute the value on every internal wire;
3. accept if and only if the final output is 1.

Because the circuit is acyclic, such a topological order exists. Each gate is evaluated once, so the verifier runs in polynomial time.

Which Certificate Is Easier?

The textbook's certificate, giving values on all wires, makes the proof easier: the verifier only checks local consistency gate by gate. A satisfying input assignment is shorter, but the verifier must evaluate the circuit in a valid order.

5 Exercise 34.3-5

Question 34.3-5

The proof of Lemma 34.6 assumes that the working storage for verifier A occupies a contiguous region of polynomial size. Where is this assumption used? Why does it cause no loss of generality?

Answer

The assumption is used when bounding the size of a configuration and therefore the size of the circuit that simulates one computation step.

Where It Is Used

In Lemma 34.6, a configuration contains the complete state of the computation: program counter, input, certificate, working memory, and auxiliary state.

To prove the constructed circuit has polynomial size, we need each configuration to have polynomially many bits. If working storage is a contiguous polynomial-size block, then the memory portion of each configuration is clearly polynomial in $n = |x|$.

Why There Is No Loss of Generality

A polynomial-time computation can use only polynomially many memory cells in any meaningful way:

- on a Turing-machine model, the head can move only one cell per step, so in polynomially many steps it can reach only polynomially many cells;
- on a random-access model, even if addresses are large, the computation can touch only polynomially many distinct locations in polynomially many steps.

If accessed locations are scattered, represent the used memory as a polynomial-size table of address-value pairs. The transition circuit updates this table. Thus the configuration can still be encoded with polynomially many bits, and the proof goes through.

6 Exercise 34.3-6**Question 34.3-6**

A language L is complete for a language class $\mathcal{C}$ with respect to polynomial-time reductions if $L \in \mathcal{C}$ and $L' \leq_p L$ for all $L' \in \mathcal{C}$. Show that $\emptyset$ and $\{0, 1\}^*$ are the only languages in $\mathbf{P}$ that are not complete for $\mathbf{P}$.

Answer

The languages $\emptyset$ and $\{0, 1\}^*$ cannot be complete for $\mathbf{P}$. Every other language in $\mathbf{P}$ is complete for $\mathbf{P}$.

 $\emptyset$ and $\{0, 1\}^*$ Are Not Complete

The language $\{0, 1\}^*$ is in $\mathbf{P}$. If $\emptyset$ were complete for $\mathbf{P}$, then we would have

$$\{0, 1\}^* \leq_p \emptyset.$$

That would require a function f such that

$$x \in \{0, 1\}^* \iff f(x) \in \emptyset.$$

The left side is always true, while the right side is always false, impossible. Thus $\emptyset$ is not complete for $\mathbf{P}$.

Similarly, $\emptyset \in \mathbf{P}$. If $\{0, 1\}^*$ were complete for $\mathbf{P}$, then

$$\emptyset \leq_p \{0, 1\}^*.$$

That would require

$$x \in \emptyset \iff f(x) \in \{0, 1\}^*.$$

The left side is always false, while the right side is always true, impossible. Thus $\{0, 1\}^*$ is not complete for $\mathbf{P}$.

Every Other Language in $\mathbf{P}$ Is Complete for $\mathbf{P}$

Let $L \in \mathbf{P}$ with

$$L \neq \emptyset \quad \text{and} \quad L \neq \{0, 1\}^*.$$

Then choose fixed strings

$$a \in L \quad \text{and} \quad b \notin L.$$

Let L' be any language in $\mathbf{P}$. Since $L' \in \mathbf{P}$, it has a polynomial-time decider A' . Define

$$f(x) = \begin{cases} a, & \text{if } A'(x) \text{ accepts,} \\ b, & \text{if } A'(x) \text{ rejects.} \end{cases}$$

The function f is polynomial-time computable because A' is polynomial-time and a, b are fixed constants. Moreover,

$$x \in L' \iff f(x) \in L.$$

Thus $L' \leq_p L$. Since L' was arbitrary, every nontrivial language $L \in \mathbf{P}$ is complete for $\mathbf{P}$.

7 Bonus: Exercise 34.3-7

Question 34.3-7

Show the complement relationship between $\mathbf{NP}$ -complete and $\mathbf{co-NP}$ -completeness.

Answer

If L is $\mathbf{NP}$ -complete, then $\bar{L}$ is complete for $\mathbf{co-NP}$. Conversely, if $\bar{L}$ is complete for $\mathbf{co-NP}$, then L is $\mathbf{NP}$ -complete.

Proof

Assume L is $\mathbf{NP}$ -complete. Then $L \in \mathbf{NP}$, so $\bar{L} \in \mathbf{co-NP}$.

Let S be any language in $\mathbf{co-NP}$. Then $\bar{S} \in \mathbf{NP}$. Since L is $\mathbf{NP}$ -complete, there is a polynomial-time reduction f such that

$$\bar{S} \leq_p L.$$

Thus, for all x ,

$$x \in \bar{S} \iff f(x) \in L.$$

Taking complements,

$$x \in S \iff f(x) \in \bar{L}.$$

So $S \leq_p \bar{L}$. Therefore $\bar{L}$ is $\mathbf{co-NP}$ -complete.

The converse follows by applying the same argument to complements.

Careful

The complement of an $\mathbf{NP}$ -complete language is known to be $\mathbf{co-NP}$ -complete, not necessarily $\mathbf{NP}$ -complete. It would be $\mathbf{NP}$ -complete only if $\mathbf{NP} = \mathbf{co-NP}$.

8 Bonus: Exercise 34.3-8

Question 34.3-8

In the proof that CIRCUIT-SAT is **NP**-hard, the reduction for a language $L \in \mathbf{NP}$ uses a verifier A for L . Explain why this is valid even though the reduction algorithm appears to depend on A .

Answer

To prove CIRCUIT-SAT is **NP**-hard, we do not need one uniform reduction algorithm that works for all languages in **NP** at once. For each fixed language $L \in \mathbf{NP}$, we only need to prove that some polynomial-time reduction from L to CIRCUIT-SAT exists.

Explanation

If $L \in \mathbf{NP}$, then by definition there exists a polynomial-time verifier $A(x, y)$ for L . Once L is fixed, this verifier A is fixed as well. The reduction algorithm may use the fixed code of A to construct the circuit that simulates $A(x, y)$.

The reduction is still polynomial time because the verifier A runs in polynomial time and has fixed program size independent of the input x .

Therefore, for every fixed $L \in \mathbf{NP}$, there exists a polynomial-time reduction

$$L \leq_p \text{CIRCUIT-SAT}.$$

That is exactly what **NP**-hard requires.

Final Summary

What to Remember

- Figure 34.8(b) is unsatisfiable because one conjunct forces $x_3 = 1$ and another forces $x_3 = 0$.
- Polynomial-time reductions are transitive.
- $L \leq_p \bar{L}$ iff $\bar{L} \leq_p L$.
- A satisfying input assignment can certify CIRCUIT-SAT, but all-wire values make local gate checking easier.
- The contiguous-memory assumption in Lemma 34.6 is used to bound configuration size; scattered memory can be compactly encoded.
- The only non-**P**-complete languages in **P** are $\emptyset$ and $\{0, 1\}^*$.